

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

SCHOOL OF COMPUTING
SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH ACADEMY
(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401

Exercise No. 14. Program to implement disk scheduling algorithms.

Objectives:

To Simulate of disk scheduling techniques.

Prerequisite / Tools Required:

Knowledge of disk scheduling algorithms.

Theory/ Concept:

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and large disk bandwidth. Both the access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced which is called as disk scheduling.

Different algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK

Procedure \ Algorithm:

Step 1: Let Request array represents an array storing indexes of tracks that have been requested in ascending order of their time of arrival. 'head' is the position of disk head. Step 2: Let us one by one take the tracks in default order and calculate the absolute distance of the track from the head.

Step 3: Increment the total seek count with this distance.

Step 4: Currently serviced track position now becomes the new head position. Step 5: Go to step 2 until all tracks in request array have not been serviced.

- Calculate the seek time as per the algorithms listed below:
- First-in-First-Out
- Shortest Seek Time First
- Scan
- C-look

Sample Input:

A disk has **200 tracks (0–199)**.
The disk request queue is: 98, 183, 37, 122, 14, 124, 65, 67

Calculate the **total head movement** using:

1. FCFS
2. SSTF
3. SCAN
4. C-look

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

#define N 8
#define MAX 199

int a[N] = {98, 183, 37, 122, 14, 124, 65, 67};

void sort(int b[]) {
    for (int i = 0; i < N - 1; i++)
        for (int j = 0; j < N - i - 1; j++)
            if (b[j] > b[j + 1]) {
                int t = b[j];
                b[j] = b[j + 1];
                b[j + 1] = t;
            }
}

void result(char name[], int seq[], int n, int head) {
    int total = 0, cur = head;

    printf("\n%s\nSequence: %d", name, head);

    for (int i = 0; i < n; i++) {
        total += abs(cur - seq[i]);
        cur = seq[i];
        printf(" -> %d", cur);
    }

    printf("\nTotal head movement: %d", total);
    printf("\nAverage head movement: %.2f\n", (float)total / N);
}

int main() {
    int head = 53, b[N], seq[2 * N + 2], n, i, k;
    int done[N] = {0}, cur, best, dist, total;

    printf("Disk tracks: 0-%d\n", MAX);
    printf("Request queue: ");
    for (i = 0; i < N; i++)
        printf("%d ", a[i]);

    printf("\nInitial head position (assumed): %d\n", head);

    // FCFS
    result("FCFS", a, N, head);
```

```
// SSTF
for (i = 0; i < N; i++) done[i] = 0;
cur = head;
total = 0;
printf("\nSSTF\nSequence: %d", head);

for (i = 0; i < N; i++) {
    best = -1;
    dist = 999999;

    for (int j = 0; j < N; j++)
        if (!done[j] && abs(a[j] - cur) < dist) {
            dist = abs(a[j] - cur);
            best = j;
        }

    done[best] = 1;
    total += dist;
    cur = a[best];
    printf(" -> %d", cur);
}

printf("\nTotal head movement: %d", total);
printf("\nAverage head movement: %.2f\n", (float)total / N);

// Sort requests
for (i = 0; i < N; i++) b[i] = a[i];
sort(b);

// SCAN: move toward 0 first
n = 0;
for (i = N - 1; i >= 0; i--)
    if (b[i] < head) seq[n++] = b[i];
seq[n++] = 0;
for (i = 0; i < N; i++)
    if (b[i] >= head) seq[n++] = b[i];
result("SCAN (toward 0 first)", seq, n, head);

// C-SCAN: move upward first, wrap from MAX to 0
n = 0;
for (i = 0; i < N; i++)
    if (b[i] >= head) seq[n++] = b[i];
seq[n++] = MAX;
seq[n++] = 0;
for (i = 0; i < N; i++)
    if (b[i] < head) seq[n++] = b[i];
```

```
    result("C-SCAN (upward first)", seq, n, head);

    // LOOK: move upward first, then reverse
    n = 0;
    for (i = 0; i < N; i++)
        if (b[i] >= head) seq[n++] = b[i];
    for (i = N - 1; i >= 0; i--)
        if (b[i] < head) seq[n++] = b[i];
    result("LOOK (upward first)", seq, n, head);

    // C-LOOK: move upward, jump to lowest pending request
    n = 0;
    for (i = 0; i < N; i++)
        if (b[i] >= head) seq[n++] = b[i];
    for (i = 0; i < N; i++)
        if (b[i] < head) seq[n++] = b[i];
    result("C-LOOK (upward first)", seq, n, head);
    return 0;}
```

```
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp14$ gcc exp14.c
rk@fedora:/mnt/shared/NOTES/Notes S-5/OS lab/OS-Lab/exp14$ ./a.out
Disk tracks: 0-199
Request queue: 98 183 37 122 14 124 65 67
Initial head position (assumed): 53

FCFS
Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total head movement: 640
Average head movement: 80.00

SSTF
Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movement: 236
Average head movement: 29.50

SCAN (toward 0 first)
Sequence: 53 -> 37 -> 14 -> 0 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183
Total head movement: 236
Average head movement: 29.50

C-SCAN (upward first)
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total head movement: 382
Average head movement: 47.75

LOOK (upward first)
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
Total head movement: 299
Average head movement: 37.38

C-LOOK (upward first)
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37
Total head movement: 322
Average head movement: 40.25
```

RESULT:

The disk scheduling algorithms were implemented and compared by calculating total head movement and average head movement for the given request queue.